



Politechnika Wrocławska



Wydział Informatyki
i Telekomunikacji

Algorytmy i złożoność obliczeniowa

Projekt 1 – Algorytmy sortowania

Huzhva Mariia, 288378

Prowadzący:
mgr inż. Damian Mroziński

Spis treści

1	Wstęp	3
2	Założenia projektu	3
3	Organizacja programu	4
3.1	Struktura projektu	4
3.2	Tryby pracy programu	5
3.3	Obsługa parametrów	5
3.4	Budowanie programu	5
4	Implementacja	6
4.1	Szablony, <code>const</code> i przekazywanie przez referencję	6
4.2	Zaimplementowane struktury danych	7
4.2.1	Tablica dynamiczna	7
4.2.2	Lista jednokierunkowa	7
4.2.3	Lista dwukierunkowa	7
4.2.4	Stos	8
4.2.5	Drzewo binarne	8
4.3	Zaimplementowane algorytmy sortowania	9
4.3.1	Bucket Sort	9
4.3.2	Quick Sort	10
4.3.3	Shell Sort	11
4.4	Operacje na plikach	12
4.5	Generowanie danych testowych	12
4.5.1	Zmiana metodologii benchmarku	13
4.5.2	Rodzaje przygotowywanych danych	13
4.5.3	Generator losowy i rozkłady	13
4.5.4	Losowanie dla napisów i znaków	14
4.5.5	Dane rosnące i malejące	14
4.5.6	Dane częściowo uporządkowane	14
4.5.7	Ziarno generatora liczb losowych	15
4.6	Weryfikacja poprawności sortowania	15
4.7	Obsługa błędów bez <code>try-catch</code>	15
5	Metodyka badań	16
5.1	Środowisko testowe	16
5.2	Sposób pomiaru czasu	16
5.3	Liczba powtórzeń i zakres badań	17
5.4	Ograniczenia środowiska badawczego	17
6	Wyniki badań i analiza	17
6.1	Badanie α – wpływ parametrów algorytmu	17
6.1.1	Quick Sort	18
6.1.2	Shell Sort	18
6.1.3	Wniosek z badania α	19
6.2	Badanie A – wpływ liczebności zbioru	19
6.2.1	Quick Sort	20

6.2.2	Shell Sort	22
6.2.3	Bucket Sort	23
6.2.4	Wniosek z badania A	25
6.3	Badanie B – wpływ rozkładu danych	25
6.3.1	Wniosek z badania B	26
6.4	Badanie C – wpływ typu danych	27
6.4.1	Wniosek z badania C	28
6.5	Badanie Ω – użycie nieliniowych struktur danych	28
6.5.1	Wniosek z badania Ω	30
7	Porównanie dwóch metodologii benchmarku	30
7.1	Tabela porównawcza	30
7.2	Przykładowe polecenia uruchomienia programu	31
8	Repozytorium GitHub	31
9	Wnioski	31

1 Wstęp

Celem projektu było stworzenie i zaimplementowanie różnych algorytmów sortowania spełniających wymagania projektowe oraz przeprowadzenie analizy ich działania i efektywności. Najważniejszym aspektem projektu było jednak nauczenie się rozumienia wymagań, samodzielnego poszukiwania rozwiązań, rozwijania myślenia analitycznego oraz zadawania pytań pozwalających lepiej zrozumieć problem.

W projekcie zaimplementowano trzy wymagane algorytmy sortowania:

- sortowanie kubelkowe (**Bucket Sort**),
- sortowanie szybkie (**Quick Sort**),
- sortowanie Shella (**Shell Sort**).

Zaimplementowano również trzy wymagane struktury liniowe:

- tablicę dynamiczną,
- listę jednokierunkową,
- listę dwukierunkową.

Na potrzeby badania Ω przygotowano dodatkowo:

- stos,
- drzewo binarne.

Projekt został wykonany w języku C++17, z ręcznym zarządzaniem pamięcią i bez użycia gotowych kontenerów standardowych takich jak `std::vector` czy `std::list`. Zgodnie z założeniami zadania, uporządkowanie elementów jest w porządku rosnącym. Program został przygotowany w taki sposób, aby umożliwiał zarówno sortowanie danych z pojedynczego pliku, jak i przeprowadzanie powtarzalnych badań benchmarkowych, w których każda iteracja może być wykonywana na nowo przygotowanych danych wejściowych. Dodatkowo zaimplementowano zapis wyników do pliku CSV.

Projekt ma charakter obiektowy, wykorzystuje szablony, został przygotowany z użyciem systemu budowania CMake i spełnia wymagania dotyczące kompilacji z flagami `-Wall` `-Wextra` `-Werror`. Dzięki temu możliwe było uruchamianie programu dla różnych kombinacji algorytmów, struktur danych, typów elementów oraz parametrów badań.

2 Założenia projektu

Podczas realizacji projektu uwzględniono następujące założenia:

1. sortowanie oznacza uporządkowanie elementów rosnąco,
2. podstawowym typem danych jest `int`,
3. wszystkie struktury danych zostały zaimplementowane ręcznie,
4. pamięć jest alokowana i zwalniana dynamicznie,

5. projekt został napisany w standardzie C++17,
6. kompilacja odbywa się przy użyciu `CMake`,
7. poprawność sortowania jest weryfikowana po wykonaniu algorytmu,
8. benchmark wykonuje wiele iteracji i oblicza minimum, maksimum oraz średni czas,
9. czas sortowania jest mierzony w mikrosekundach,
10. do czasu benchmarku nie wlicza się generowania danych, ładowania plików ani zapisu wyników,
11. wyniki badań są zapisywane do pliku `CSV`,
12. program działa w oparciu o argumenty przekazywane do `main`,
13. do obsługi parametrów użyto biblioteki udostępnionej przez prowadzącego,
14. w typowych i przewidywalnych sytuacjach błędnych nie stosowano `try-catch`.

Dodatkowo, zgodnie z wymaganiami na ocenę 5.0:

- wykorzystano trzy wymagane algorytmy sortowania,
- wykorzystano trzy wymagane struktury liniowe,
- przygotowano badanie wpływu parametrów algorytmu,
- przygotowano badanie wpływu liczebności zbioru,
- przygotowano badanie wpływu rozkładu danych,
- przygotowano badanie wpływu typu danych,
- przygotowano badanie z użyciem struktur nieliniowych,
- zastosowano podejście obiektowe,
- zastosowano szablony.

3 Organizacja programu

3.1 Struktura projektu

Projekt został podzielony na trzy podstawowe części:

- `src/` — pliki źródłowe implementujące logikę programu,
- `include/` — pliki nagłówkowe z deklaracjami klas, struktur i funkcji,
- `lib/` — zewnętrzna biblioteka do obsługi parametrów dostarczona przez prowadzącego.

3.2 Tryby pracy programu

Program obsługuje trzy podstawowe tryby:

- **singleFile** - służy do sortowania danych z pojedynczego pliku wejściowego(.txt) i zapisu wyniku do pliku wyjściowego(.txt).
- **benchmark** - umożliwia wykonywanie wielokrotnych pomiarów czasu dla konkretnej konfiguracji badania.
- **help** - wyświetla listę obsługiwanych parametrów i sposób użycia programu.

3.3 Obsługa parametrów

Sterowanie programem odbywa się przez argumenty przekazywane do funkcji **main**. Do interpretacji parametrów użyto biblioteki udostępnionej przez prowadzącego. Dzięki temu zachowano zgodność z wymaganym interfejsem uruchamiania programu.

Parametry określają między innymi:

- tryb pracy,
- algorytm sortowania,
- strukturę danych,
- typ danych,
- rozkład danych,
- rozmiar struktury,
- liczbę iteracji,
- wybór pivota dla Quick Sort,
- wariant odstępów dla Shell Sort,
- plik wejściowy, wyjściowy i plik z wynikami badań.

3.4 Budowanie programu

Projekt budowany jest za pomocą **CMake**. W pliku **CMakeLists.txt** ustawiono standard **C++17** oraz włączono flagi ostrzeżeń:

```
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

target_compile_options(Project PRIVATE
    -Wall
    -Wextra
    -Werror
)
```

Takie ustawienie wymusza zgodność ze standardem oraz eliminuje możliwość pozostawienia ostrzeżeń kompilatora bez reakcji. Jest to istotne, ponieważ projekt powinien kompilować się bez ostrzeżeń.

4 Implementacja

4.1 Szablony, const i przekazywanie przez referencję

Jedną z istotnych decyzji projektowych było szerokie wykorzystanie szablonów. Dzięki temu ta sama logika mogła zostać zastosowana do różnych typów danych oraz różnych struktur, bez konieczności przepisywania całych algorytmów osobno dla każdego przypadku. Należy jednak zaznaczyć, że ze względu na różnice w budowie i sposobie dostępu do elementów nie wszystkie struktury mogły być obsługiwane w identyczny sposób. W praktyce łatwiej było uogólnić wspólne mechanizmy dla list, natomiast tablica nie mogła zostać ujęta dokładnie w ten sam schemat.

Przykładowo, klasa tablicy została przygotowana jako:

```
template <typename T>
class Array {
private:
    T* data;
    int size;

public:
    explicit Array(int s) { ... }
    ~Array() { delete[] data; }

    int getSize() const { return size; }
    bool empty() const { return size == 0; }

    bool get(int index, T& value) const { ... }
    bool set(int index, const T& value) { ... }

    Array(const Array&) = delete;
    Array& operator=(const Array&) = delete;
};
```

W powyższym fragmencie widać kilka ważnych decyzji:

- `template <typename T>` pozwala używać tej samej struktury dla różnych typów,
- `getSize() const` oznacza, że metoda nie zmienia stanu obiektu,
- `get(int index, T& value) const` odczytuje wartość bez kopiowania całej struktury i bez modyfikowania obiektu,
- `set(int index, const T& value)` przyjmuje wartość przez stałą referencję, aby uniknąć niepotrzebnego kopiowania,
- usunięcie konstruktora kopiującego i operatora przypisania chroni przed przypadkowym skopiowaniem obiektu zarządzającego surowym wskaźnikiem.

Szczególnie ważne jest rozróżnienie między:

- `const T&` — obiekt wejściowy jest przekazywany bez kopiowania i nie może zostać zmieniony wewnątrz funkcji,
- `T&` — referencja nie stała, wykorzystywana najczęściej jako parametr wyjściowy,

- `const Structure&` — przekazanie całej struktury do odczytu bez tworzenia kopii.

Takie podejście poprawia wydajność i zwiększa bezpieczeństwo implementacji.

4.2 Zaimplementowane struktury danych

4.2.1 Tablica dynamiczna

Struktura `Array<T>` przechowuje wskaźnik na dynamicznie zaalokowaną tablicę oraz liczbę elementów. Udostępnia operacje:

- `getSize()`,
- `empty()`,
- `get(index, value)`,
- `set(index, value)`,
- `clear()`.

Tablica jest strukturą naturalnie dostosowaną do algorytmów opartych na częstym dostępie indeksowanym. Odczyt i zapis elementu pod danym indeksem mają tu koszt stały.

4.2.2 Lista jednokierunkowa

Lista jednokierunkowa została zaimplementowana ręcznie jako łańcuch węzłów, z których każdy przechowuje wartość oraz wskaźnik na następny element. Udostępniono między innymi:

- `pushFront(...)` ,
- `pushBack(...)` ,
- `get(index, value)` ,
- `set(index, value)` ,
- `clear()` .

Dostęp do elementu po indeksie wymaga przejścia od początku listy do żądanej pozycji. Oznacza to, że operacje oparte na indeksach są znacznie wolniejsze niż w tablicy.

4.2.3 Lista dwukierunkowa

Struktura `DoubleList<T>` składa się z dynamicznie alokowanych węzłów połączonych ze sobą w obu kierunkach. Każdy węzeł przechowuje wartość oraz wskaźniki na element poprzedni i następny, a cała struktura dodatkowo przechowuje liczbę elementów. Podobnie jak pozostałe zaimplementowane struktury udostępnia ona zbliżony zestaw podstawowych operacji :

- `getSize()`,
- `empty()`,

- `get(index, value)`,
- `set(index, value)`,
- `clear()`.

Dzięki zachowaniu podobnego interfejsu możliwe było wykorzystanie szablonów i ponowne użycie tej samej logiki w różnych strukturach.

Jednocześnie lista dwukierunkowa różni się od tablicy pod względem sposobu dostępu do elementów. Mimo dodatkowego wskaźnika dostęp do elementu o danym indeksie nadal wymaga przejścia przez kolejne węzły, dlatego nie odbywa się w czasie stałym. Cecha ta wpływa na efektywność działania algorytmów sortowania uruchamianych na tej strukturze.

4.2.4 Stos

Stos został zaimplementowany jako lista połączonych węzłów ze wskaźnikiem na szczyt struktury. Podstawowe operacje to:

- `push(...)` – dodanie nowego elementu na szczyt stosu,
- `pop(...)` – usunięcie elementu znajdującego się na szczycie stosu,
- `top(...)` – odczyt wartości elementu znajdującego się na szczycie bez jego usuwania,
- `get(index,...)` – pomocniczy odczyt elementu z wybranej pozycji,
- `set(index,...)` – pomocnicza modyfikacja elementu na wybranej pozycji.

Metody `get(index,...)` oraz `set(index,...)` zostały dodane pomocniczo, aby możliwe było wykorzystanie tej samej logiki benchmarku i części algorytmów także dla stosu. Nie zmienia to jednak podstawowego charakteru tej struktury, która nadal działa zgodnie z zasadą LIFO.

4.2.5 Drzewo binarne

Drzewo binarne zostało zaimplementowane jako pełne drzewo binarne wypełniane poziomami. Węzeł przechowuje:

- wartość,
- wskaźnik na lewe dziecko,
- wskaźnik na prawe dziecko.

Struktura udostępnia operacje:

- `pushBack(...)` ,
- `get(index,...)` ,
- `set(index,...)` ,
- `clear()` .

Drzewo zostało wykorzystane w badaniu Ω do porównania struktur liniowych z nieliniowymi.

4.3 Zaimplementowane algorytmy sortowania

4.3.1 Bucket Sort

Bucket Sort został zaimplementowany w postaci szablonowej i działa dla struktur:

- `Array<T>`,
- `SingleList<T>`,
- `DoubleList<T>`.

Wspólna logika algorytmu została oparta na funkcjach pomocniczych:

- `readAt(...)` – odczyt elementu ze struktury,
- `writeAt(...)` – zapis elementu do struktury,
- `insertSorted(...)` – wstawienie elementu do kubelka w odpowiednim miejscu,
- `sortImpl(...)` – właściwa funkcja realizująca sortowanie kubelkowe,
- `BucketTraits<T>` – pomocniczy mechanizm wyznaczania indeksu kubelka dla danego typu.

Najważniejszym elementem implementacji było oddzielenie ogólnej logiki Bucket Sort od sposobu wyznaczania numeru kubelka. Dla typów liczbowych indeks kubelka jest liczony na podstawie wartości minimalnej, maksymalnej oraz zakresu danych. Przykładowo:

```
long double normalized =  
    (static_cast<long double>(value) - minAsLongDouble) / range;  
  
int index = static_cast<int>(normalized * (bucketCount - 1));
```

Takie rozwiązanie działa dla typów liczbowych, ponieważ można obliczyć różnicę między wartościami. Inaczej wygląda sytuacja dla typu `std::string`. Dla napisów nie można wykonać operacji odejmowania, dlatego przygotowano osobną specjalizację `BucketTraits<std::string>`.

```
template <>  
struct BucketTraits<std::string> {  
    static int bucketCount(int) {  
        return 257;  
    }  
  
    static int getBucketIndex(  
        const std::string& value,  
        const std::string& ,  
        const std::string& ,  
        int bucketCount  
    ) {  
        if (value.empty()) {  
            return 0;  
        }  
    }  
};
```

```

    int index = static_cast<int>(
        static_cast<unsigned char>(value[0])
    ) + 1;

    if (index >= bucketCount) {
        index = bucketCount - 1;
    }

    return index;
}
};

```

W tej wersji dla typu `std::string` kubełek jest wybierany na podstawie pierwszego znaku napisu. Sam znak jest traktowany jako kod znaku. Napisy zaczynające się od tej samej litery trafiają więc do tego samego kubełka, a wewnątrz kubełka są sortowane leksykograficznie.

Dzięki temu główna część Bucket Sort pozostaje wspólna i szablonowa dla wszystkich obsługiwanych typów, a różni się jedynie sposób wyznaczania indeksu kubełka.

4.3.2 Quick Sort

Quick Sort został zaimplementowany w postaci szablonowej i działa dla każdej struktury, która korzysta z funkcji pomocniczych:

- `getValue(...)` — odczyt elementu,
- `swapAt(...)` — zamiana dwóch elementów,
- `QuickSorting(...)` — właściwa funkcja rekurencyjna.

Istotny fragment odpowiedzialny za wybór pivota:

```

static int choosePivotIndex(int left, int right, Parameters::Pivots pivotType) {
    if (pivotType == Parameters::Pivots::left) {
        return left;
    }

    if (pivotType == Parameters::Pivots::right) {
        return right;
    }

    if (pivotType == Parameters::Pivots::random) {
        static std::mt19937 gen(std::random_device{}());
        std::uniform_int_distribution<int> dist(left, right);
        return dist(gen);
    }

    return left + (right - left) / 2;
}

```

W badaniu α porównano trzy warianty wyboru pivota:

- losowy,

- środkowy,
- skrajny.

Z punktu widzenia badań wystarczyło uwzględnienie jednego wariantu skrajnego. W implementacji przygotowano jednak zarówno pivot lewy, jak i pivot prawy, aby interfejs programu był pełniejszy. W badaniu α wariant skrajny był reprezentowany przez jeden wybrany przypadek skrajny.

4.3.3 Shell Sort

Shell Sort również został zaimplementowany jako algorytm szablonowy. Wspólna logika została rozbita na funkcje pomocnicze:

- `readAt(...)` ,
- `writeAt(...)` ,
- `insertion(...)` ,
- `shellHalfGaps(...)` ,
- `shellKnuthGaps(...)` ,
- `sortImpl(...)` .

Przykład implementacji dwóch wariantów odstępów:

```
template <typename Structure, typename T>
static void shellHalfGaps(Structure& structure) {
    for (int gap = structure.getSize() / 2; gap > 0; gap /= 2) {
        insertion<Structure, T>(structure, gap);
    }
}

template <typename Structure, typename T>
static void shellKnuthGaps(Structure& structure) {
    int gap = 1;
    while (gap < structure.getSize() / 3) {
        gap = 3 * gap + 1;
    }

    while (gap >= 1) {
        insertion<Structure, T>(structure, gap);
        gap = (gap - 1) / 3;
    }
}
```

W badaniu α porównano:

- `option1` — odstępów dzielone przez 2,
- `option2` — ciąg Knutha.

4.4 Operacje na plikach

Obsługa plików została wydzielona do osobnej przestrzeni nazw `FileHandler`. Takie rozwiązanie pozwoliło oddzielić logikę wejścia i wyjścia od właściwej logiki algorytmów sortowania oraz od kodu odpowiedzialnego za benchmarki.

W projekcie zaimplementowano trzy główne grupy operacji związanych z plikami:

- wczytywanie danych do wybranej struktury z pliku wejściowego,
- zapisywanie posortowanych danych do pliku wyjściowego,
- zapisywanie wyników badań do pliku CSV.

W trybie `singleFile` program odczytuje dane z pliku tekstowego, tworzy na ich podstawie odpowiednią strukturę danych, wykonuje sortowanie, a następnie może zapisać wynik do kolejnego pliku. Z kolei w trybie `benchmark` pliki wejściowe z danymi nie są używane, natomiast końcowe wyniki pomiarów są dopisywane do pliku CSV.

Format pliku wejściowego i wyjściowego jest zgodny z wymaganiami zadania. W pierwszej linii znajduje się liczba elementów, a w kolejnych liniach zapisane są poszczególne wartości. Przykładowo, plik wejściowy dla pięciu elementów może wyglądać następująco:

```
5
76
-142
0
2138
-4
```

Warto podkreślić, że sposób wczytywania danych został dopasowany do różnych typów elementów. Dla większości typów wykorzystywany jest standardowy operator `»`, natomiast dla typu `std::string` zastosowano osobną obsługę opartą na `getline`, aby możliwe było poprawne odczytywanie całych napisów.

4.5 Generowanie danych testowych

Za przygotowanie danych wejściowych do benchmarków odpowiada klasa `RandomArrayGenerator`. Jej zadaniem jest tworzenie danych w różnych układach, tak aby przed każdą iteracją pomiaru można było przygotować nową strukturę testową zgodną z wybranym rozkładem. Dzięki temu możliwe było oddzielenie logiki generowania danych od logiki algorytmów sortowania i od właściwej części benchmarku.

W implementacji tej klasy wykorzystano kilka bibliotek standardowych języka C++:

- `<new>` – do bezpiecznej alokacji dynamicznej z użyciem `new` (`std::nothrow`),
- `<random>` – do generowania liczb losowych i korzystania z odpowiednich rozkładów,
- `<string>` – do obsługi typu `std::string` oraz budowania losowych napisów,
- `<limits>` – do odczytu minimalnych i maksymalnych wartości danego typu.

Dodatkowo klasa korzysta z zaimplementowanych w projekcie struktur:

- `Array`,

- `SingleList`,
- `DoubleList`,
- `Stack`,
- `BinaryTree`.

Pozwala to używać jednego wspólnego generatora danych dla różnych struktur bez powielania tej samej logiki w wielu miejscach programu.

4.5.1 Zmiana metodologii benchmarku

W początkowej wersji benchmarku zastosowano podejście polegające na utworzeniu jednej struktury bazowej, a następnie wykonywaniu jej kopii przed każdą iteracją pomiaru. Takie rozwiązanie miało zapewnić identyczne dane wejściowe we wszystkich powtórzeniach i wyeliminować problem sortowania tej samej już uporządkowanej struktury.

Po ponownej analizie wymagań projektu podejście to zostało jednak zmienione. W opisie parametrów benchmarku zaznaczono, że w kolejnych iteracjach powinny być przygotowywane nowe dane wejściowe, w szczególności dla rozkładu losowego. Z tego powodu w wersji końcowej zrezygnowano z kopiowania jednej struktury bazowej.

W końcowej implementacji przed każdą iteracją tworzona jest nowa struktura i ponownie wypełniana zgodnie z wybranym rozkładem danych. Oznacza to, że dla rozkładu `random` każda iteracja otrzymuje nowy zestaw danych losowych, natomiast dla rozkładów `ascending`, `descending` oraz `ascending50Per` odpowiedni układ danych jest odtwarzany od nowa w każdej iteracji.

Takie rozwiązanie lepiej odpowiada wymaganiom projektu oraz lepiej opisuje średnie zachowanie algorytmu dla wielu rzeczywistych przypadków wejściowych, a nie tylko dla wielokrotnie kopiowanego jednego zestawu danych.

4.5.2 Rodzaje przygotowywanych danych

Klasa `RandomArrayGenerator` umożliwia przygotowanie czterech podstawowych rozkładów danych:

- `random` – dane losowe,
- `ascending` – dane rosnące,
- `descending` – dane malejące,
- `ascending50Per` – dane częściowo uporządkowane.

4.5.3 Generator losowy i rozkłady

Do losowania wartości wykorzystano generator `std::mt19937`. Generator ten jest tworzony tylko raz i później używany przy kolejnych wywołaniach funkcji losujących. Dzięki temu rozwiązaniu nie trzeba za każdym razem inicjalizować generatora od nowa.

Dla różnych typów danych zastosowano różne rozkłady:

- dla typów całkowitych użyto `std::uniform_int_distribution`,
- dla typów zmiennoprzecinkowych użyto `std::uniform_real_distribution`,

- dla typu `char` użyto losowania w zakresie printable ASCII,
- dla typu `std::string` budowano losowe napisy z printable characters.

W przypadku typów liczbowych zakres losowania jest wyznaczany przy pomocy `std::numeric_limits`. Oznacza to, że generator korzysta z dopuszczalnego zakresu danego typu, zamiast używać na stałe wpisanych granic liczbowych. Przykładowo dla typu `int` wykorzystywany jest kod:

```
std::uniform_int_distribution<int> dist(
    std::numeric_limits<int>::min(),
    std::numeric_limits<int>::max()
);
```

Takie rozwiązanie jest bardziej ogólne i poprawne, ponieważ działa zgodnie z właściwościami konkretnego typu danych.

4.5.4 Losowanie dla napisów i znaków

Typ `char` został potraktowany osobno. Zamiast losowania całego zakresu wartości bajtowych wybrano tylko znaki printable ASCII, czyli takie, które można bez problemu zapisać i odczytać z pliku tekstowego. Dzięki temu dane wejściowe i wyjściowe pozostają czytelne.

Podobnie dla typu `std::string` tworzone losowe napisy złożone z printable characters. Najpierw losowana jest długość napisu, a następnie kolejne znaki wybierane są ze zdefiniowanego zbioru znaków. Pozwala to stworzyć poprawne i wygodne do testowania dane tekstowe.

4.5.5 Dane rosnące i malejące

Rozkłady `ascending` i `descending` nie są tworzone przez zwykłe losowanie, lecz przez kontrolowane wyznaczanie kolejnych wartości. Dzięki temu dane rzeczywiście mają oczekiwany porządek.

Dla typów liczbowych całkowitych i zmiennoprzecinkowych jest to stosunkowo proste, ponieważ kolejne elementy można wyznaczać bezpośrednio na podstawie indeksu. Inaczej wygląda sytuacja dla typów `char` oraz `unsigned char`, ponieważ ich zakres jest ograniczony.

W tych dwóch przypadkach nie zastosowano operatora `%`, ponieważ prowadziłby on do zawijania wartości po dojściu do końca zakresu. Przykładowo dla `unsigned char` po wartości 255 kolejne elementy znów przyjmowałyby wartości 0, 1, 2 i tak dalej. W efekcie ciąg, który miał być rosnący, przestałby być naprawdę uporządkowany. Z tego powodu wybrano inne rozwiązanie: po osiągnięciu granicy zakresu wartość zostaje zatrzymana na maksimum albo minimum. Dzięki temu dane zachowują poprawny układ rosnący lub malejący.

4.5.6 Dane częściowo uporządkowane

Rozkład `ascending50Per` został przygotowany jako przypadek pośredni między danymi całkowicie losowymi a całkowicie uporządkowanymi. Najpierw cała struktura jest wypełniana danymi losowymi, a następnie pierwsza połowa elementów zostaje zastąpiona wartościami rosnącymi. Dzięki temu powstaje zbiór częściowo uporządkowany.

Taki przypadek został dodany po to, aby sprawdzić, jak algorytmy zachowują się dla danych, które nie są ani całkiem przypadkowe, ani całkiem posortowane.

4.5.7 Ziarno generatora liczb losowych

W przypadku danych losowych najbardziej porównywalnym rozwiązaniem byłoby użycie tych samych wartości początkowych generatora dla każdej badanej konfiguracji. Dzięki temu każdy algorytm i każda struktura danych pracowałyby dokładnie na tych samych zestawach wejściowych, co zapewniłoby maksymalnie równe warunki porównania. W praktyce takie podejście wymagałoby jednak przygotowania i kontrolowania wielu osobnych serii danych dla kolejnych pomiarów, co znacząco wydłużyłoby czas wykonywania badań oraz skomplikowało cały benchmark. Z tego względu zdecydowano się na rozwiązanie bardziej praktyczne, przy zachowaniu dużej liczby powtórzeń, co pozwoliło uzyskać wyniki wystarczająco wiarygodne do porównania ogólnych tendencji.

4.6 Weryfikacja poprawności sortowania

Po każdym wykonaniu sortowania uruchamiana jest kontrola poprawności wyniku przez klasę `SortingCheck`. Mechanizm ten przechodzi po strukturze od początku do końca i porównuje każdy element z poprzednim.

Reprezentatywny fragment:

```
for (int i = 1; i < structure.getSize(); ++i) {
    T last{};
    T current{};

    if (!structure.get(i - 1, last) || !structure.get(i, current)) {
        return false;
    }

    if (current < last) {
        return false;
    }
}
```

Sprawdzanie poprawności jest konieczne, ponieważ sam pomiar czasu nie ma znaczenia, jeśli algorytm zwraca błędny wynik.

4.7 Obsługa błędów bez try-catch

Zgodnie z wymaganiami projektu, typowe i przewidywalne sytuacje błędne nie są obsługiwane przez try-catch. Zamiast tego zastosowano:

- sprawdzanie poprawności parametrów wejściowych,
- sprawdzanie poprawności otwierania plików,
- sprawdzanie poprawności alokacji pamięci przez `new (std::nothrow)`,
- zwracanie wartości logicznej `true/false`,
- wypisywanie komunikatów błędów na standardowe wyjście błędów.

Przykład:


```
Node* newNode = new (std::nothrow) Node(value);
if (newNode == nullptr) {
    return false;
}
```

Takie rozwiązanie jest w tym projekcie bardziej odpowiednie, ponieważ błędy takie jak brak pliku, niepoprawny parametr czy nieudana alokacja pamięci są sytuacjami spodziewanymi i powinny być kontrolowane jawnie.

5 Metodyka badań

5.1 Środowisko testowe

Badania przeprowadzono w środowisku maszyny wirtualnej z systemem Linux zgodnym z wymaganym środowiskiem przedmiotu. Program był kompilowany i uruchamiany przy użyciu CMake oraz kompilatora g++/gcc. Środowisko uruchomiono w VirtualBox.

Konfiguracja środowiska badawczego:

- system operacyjny: Linux / Xubuntu w maszynie wirtualnej,
- liczba przydzielonych rdzeni: 6,
- pamięć RAM przydzielona maszynie: około 7440 MB,
- system budowania: CMake,
- kompilacja: standard C++17.

5.2 Sposób pomiaru czasu

Do pomiaru czasu użyto biblioteki `std::chrono`. Mierzony był wyłącznie czas wykonania algorytmu sortowania. Oznacza to, że poza zakresem pomiaru pozostawały:

- tworzenie nowej struktury dla danej iteracji,
- generowanie danych zgodnie z wybranym rozkładem,
- sprawdzanie poprawności sortowania,
- zapis do pliku CSV.

Odpowiada to wymaganiom projektu i zwiększa porównywalność wyników.

Reprezentatywny fragment benchmarku:

```
const auto start = std::chrono::steady_clock::now();

if (!sortStructure(testStructure)) {
    return false;
}

const auto end = std::chrono::steady_clock::now();

const auto elapsed =
    std::chrono::duration_cast<std::chrono::microseconds>(end - start);
```

Warto podkreślić, że dane testowe były przygotowywane przed uruchomieniem pomiaru czasu, a sam zegar obejmował wyłącznie wykonanie algorytmu sortowania. Dzięki temu wyniki dotyczą bezpośrednio kosztu sortowania, a nie czasu potrzebnego na budowę struktury lub wygenerowanie danych wejściowych.

5.3 Liczba powtórzeń i zakres badań

Każdy przypadek pomiarowy wykonywano 30 razy. Następnie obliczano:

- czas minimalny,
- czas maksymalny,
- czas średni.

W analizie za podstawową wartość porównawczą przyjęto średni czas wykonania. W badaniu A przyjęto rozmiary:

1000, 2500, 5000, 7500, 10000.

Rozmiar środkowy użyty następnie w badaniach α , B, C oraz Ω wynosił:

5000.

Pełne wyniki wszystkich uruchomień zostały zapisane w plikach CSV dołączonych do archiwum z kodem. W sprawozdaniu przedstawiono głównie wartości minimalne, maksymalne i średnie oraz wykresy ułatwiające porównanie wyników.

5.4 Ograniczenia środowiska badawczego

Projekt był testowany i badany w środowisku maszyny wirtualnej. Przy bardzo dużych rozmiarach danych, szczególnie dla list i struktur opartych na dostępie indeksowanym, czasy wykonania rosły bardzo silnie. W praktyce konfiguracje z rozmiarem 20000 dla niektórych przypadków listowych okazywały się zbyt kosztowne obliczeniowo lub prowadziły do niestabilności środowiska wirtualnego.

Z tego powodu końcowy zakres badania A został dostosowany do możliwości sprzętowych i stabilności środowiska, przy zachowaniu wymogu kilku wyraźnie różnych rozmiarów. Takie podejście jest zgodne z założeniem zadania, że dobór zakresu badań powinien uwzględniać możliwości sprzętu.

6 Wyniki badań i analiza

6.1 Badanie α – wpływ parametrów algorytmu

Celem badania α było dobranie najlepszych parametrów dla algorytmów Quick Sort oraz Shell Sort przed wykonaniem dalszych pomiarów. Badanie przeprowadzono dla struktury `Array`, typu danych `int`, rozkładu `random`, rozmiaru 5000 oraz 30 powtórzeń. Dla Quick Sort porównano różne warianty wyboru pivotu, natomiast dla Shell Sort zestawiono dwa warianty ciągów odstępów.

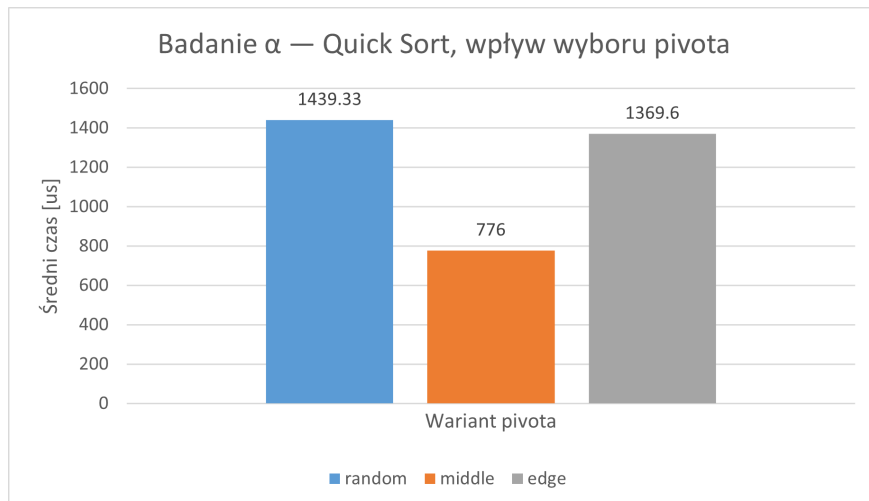
6.1.1 Quick Sort

Dla Quick Sort porównano trzy warianty wyboru pivota:

- losowy,
- środkowy,
- skrajny.

Tabela 1: Wpływ wyboru pivota na czas działania Quick Sort

Pivot	Min [μ s]	Max [μ s]	Avg [μ s]
Losowy	720	7839	1439.33
Środkowy	687	1234	776.00
Skrajny	768	7801	1369.60



Rysunek 1: Wykres dla α Quick Sort

Najlepszy wynik uzyskał pivot środkowy, dla którego średni czas wykonania wyniósł 776.00 [μ s].

Wariant losowy okazał się najwolniejszy średnio, natomiast pivot skrajny osiągnął wynik pośredni. Warto zauważyć, że pivot środkowy uzyskał nie tylko najniższy czas średni, ale również wyraźnie niższy czas maksymalny od pozostałych wariantów, co wskazuje na większą stabilność działania. Na podstawie tego badania do dalszych pomiarów wybrano Quick Sort z pivotem środkowym.

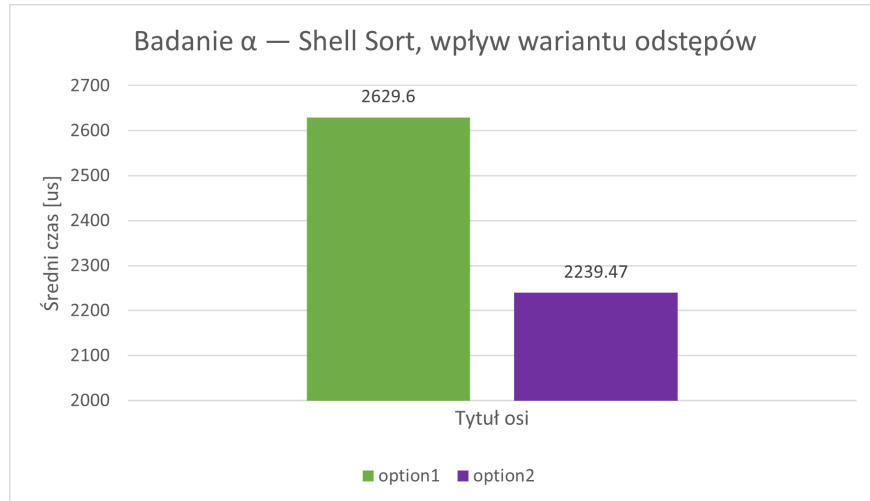
6.1.2 Shell Sort

Dla Shell Sort porównano dwa warianty odstępów:

- option1,
- option2.

Tabela 2: Wpływ wariantu odstępów na czas działania Shell Sort

Wariant	Min [μ s]	Max [μ s]	Avg [μ s]
Option1	1604	7941	2629.60
Option2	1373	7177	2239.47



Rysunek 2: Wykres dla α Shell Sort

Lepszy wynik uzyskano dla **option2**, dla którego średni czas wykonania wyniósł 2239.47 [μ s]. Dla **option1** średni czas był wyższy i wyniósł 2629.60 [μ s]. Oznacza to, że w badanej konfiguracji wariant **option2** był wydajniejszy.

Warto zauważyć, że **option2** osiągnął również niższy czas minimalny oraz niższy czas maksymalny. Świadczy to o tym, że był nie tylko szybszy średnio, ale także bardziej stabilny. Na podstawie tego badania do dalszych pomiarów wybrano Shell Sort z parametrem **option2**.

6.1.3 Wniosek z badania α

Badanie α pozwoliło wybrać najlepsze parametry dla obu analizowanych algorytmów. Dla Quick Sort najlepszy wynik uzyskał pivot środkowy, natomiast dla Shell Sort lepszy okazał się wariant **option2**. Oznacza to, że nawet przy tej samej strukturze danych, rozmiarze i liczbie elementów niewielkie zmiany w sposobie działania algorytmu mogą wyraźnie wpłynąć na czas wykonania.

6.2 Badanie A – wpływ liczebności zbioru

Celem badania A było określenie wpływu liczebności zbioru na czas działania poszczególnych kombinacji algorytmów i struktur danych. We wszystkich przypadkach użyto:

- typu danych `int`,
- rozkładu `random`,
- 30 iteracji,

- parametrów wybranych w badaniu α .

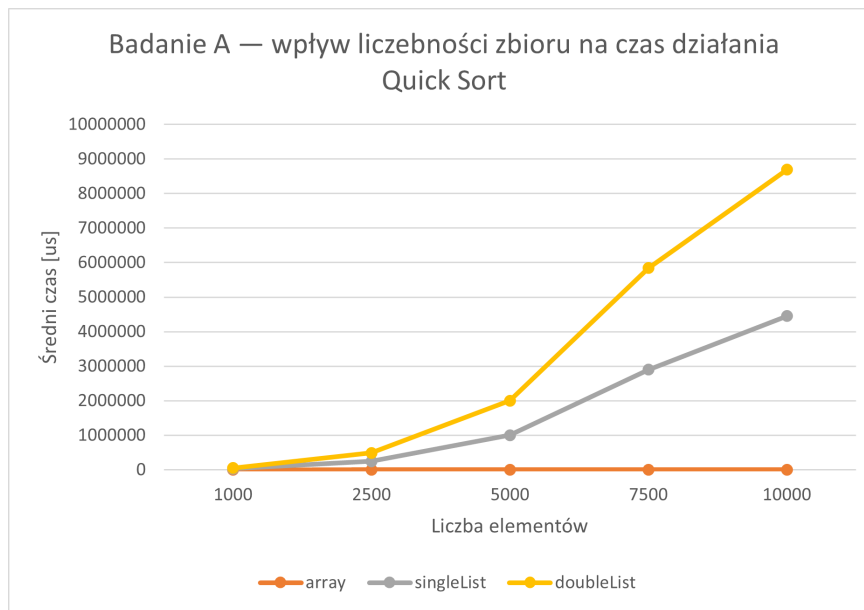
Badanie wykonano dla rozmiarów:

1000, 2500, 5000, 7500, 10000.

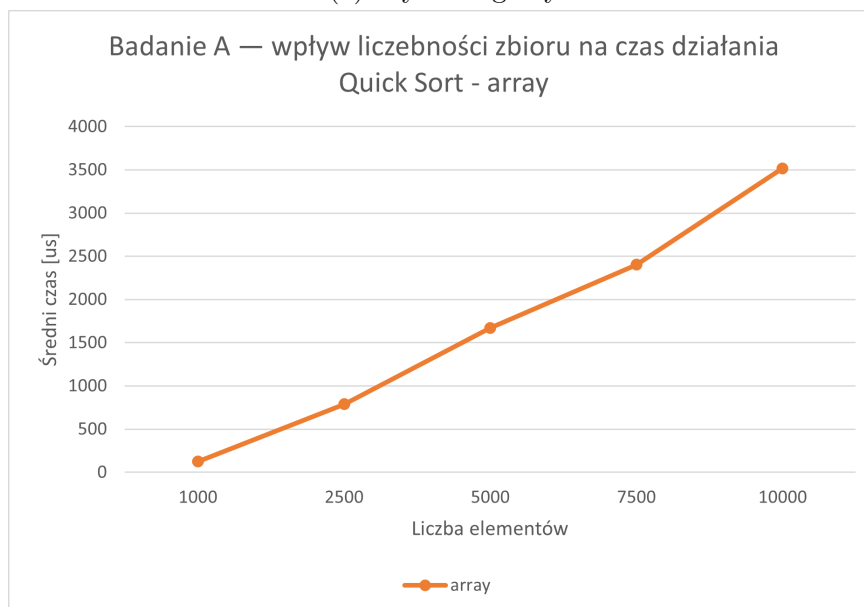
6.2.1 Quick Sort

Tabela 3: Badanie A – Quick Sort

Rozmiar	Array Avg [μ s]	SingleList Avg [μ s]	DoubleList Avg [μ s]
1000	124.93	27995.03	26280.67
2500	788.93	247976.33	237755.77
5000	1669.50	999787.77	1001803.30
7500	2401.63	2896449.77	2939838.27
10000	3517.47	4450523.13	4234025.40



(a) Wykres ogólny



(b) Powiększenie dla mniejszych wartości - Array

Rysunek 3: Badanie A – wpływ liczebności zbioru na średni czas działania Quick Sort

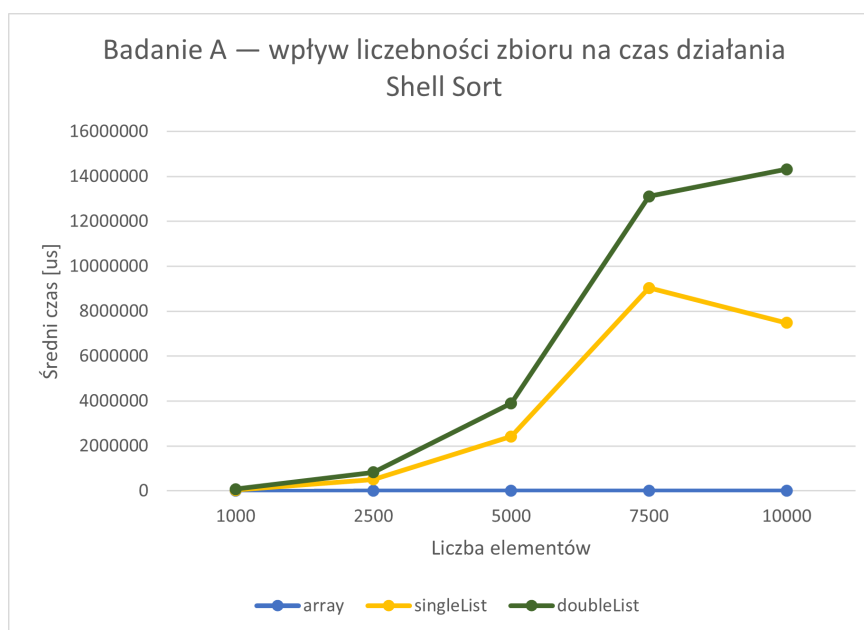
W przypadku Quick Sort wyraźnie widać, że rodzaj struktury danych miał duży wpływ na czas działania algorytmu. We wszystkich badanych przypadkach najszybsza była tablica. Wraz ze wzrostem liczby elementów czas sortowania w tablicy rósł, ale znacznie wolniej niż w listach.

Dla listy jednokierunkowej i dwukierunkowej wyniki były do siebie podobne. Różnice między nimi były niewielkie, szczególnie w porównaniu z dużą przewagą tablicy. Wynikało to z tego, że Quick Sort w tej implementacji lepiej współpracował z tablicą, ponieważ dostęp do elementów po indeksie jest w niej szybki i bezpośredni. W listach natomiast, aby dotrzeć do wybranego elementu, trzeba przechodzić przez kolejne węzły, co wydłużało czas działania algorytmu.

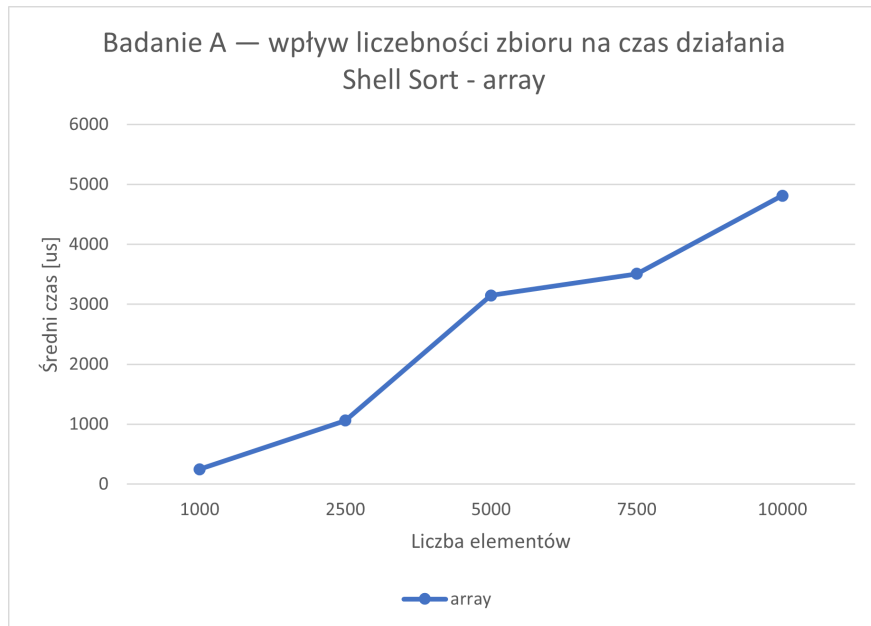
6.2.2 Shell Sort

Tabela 4: Badanie A – Shell Sort

Rozmiar	Array Avg [μ s]	SingleList Avg [μ s]	DoubleList Avg [μ s]
1000	247.53	35732.50	33395.83
2500	1061.10	508300.23	310320.83
5000	3148.73	2411650.10	1482405.70
7500	3508.80	9024904.83	4079226.37
10000	4813.53	7476424.33	6829752.53



Rysunek 4: Wykres ogólny



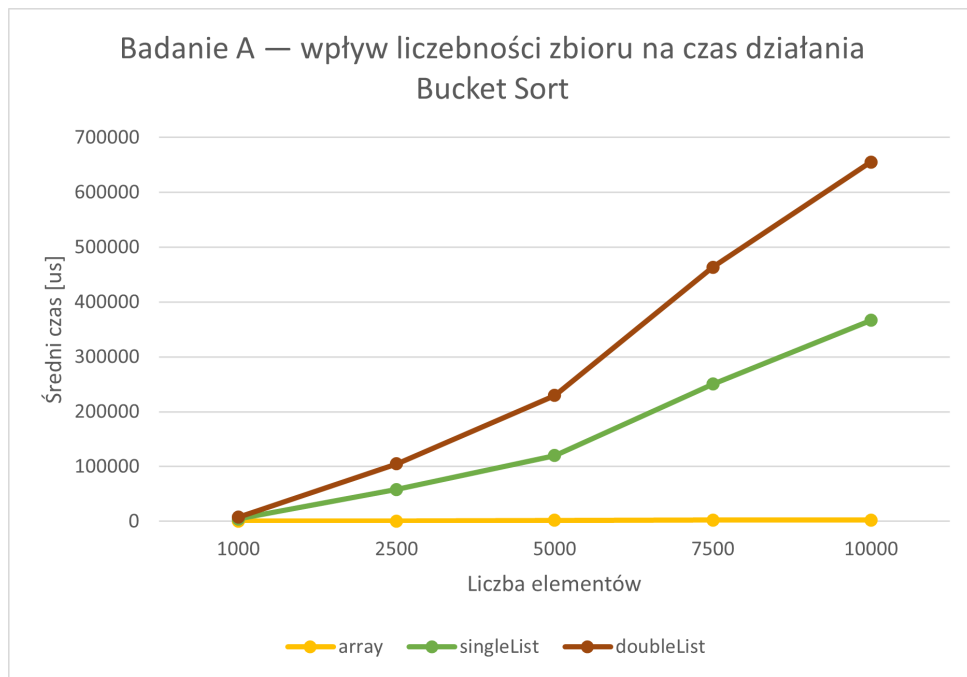
Rysunek 5: Powiększenie dla mniejszych wartości - Array

Dla Shell Sort otrzymano podobne wyniki jak dla Quick Sort. Najlepiej ponownie wypadła tablica, która dla wszystkich badanych rozmiarów miała najkrótszy czas działania. Listy dawały wyraźnie gorsze wyniki, a czas sortowania rósł w ich przypadku znacznie szybciej.

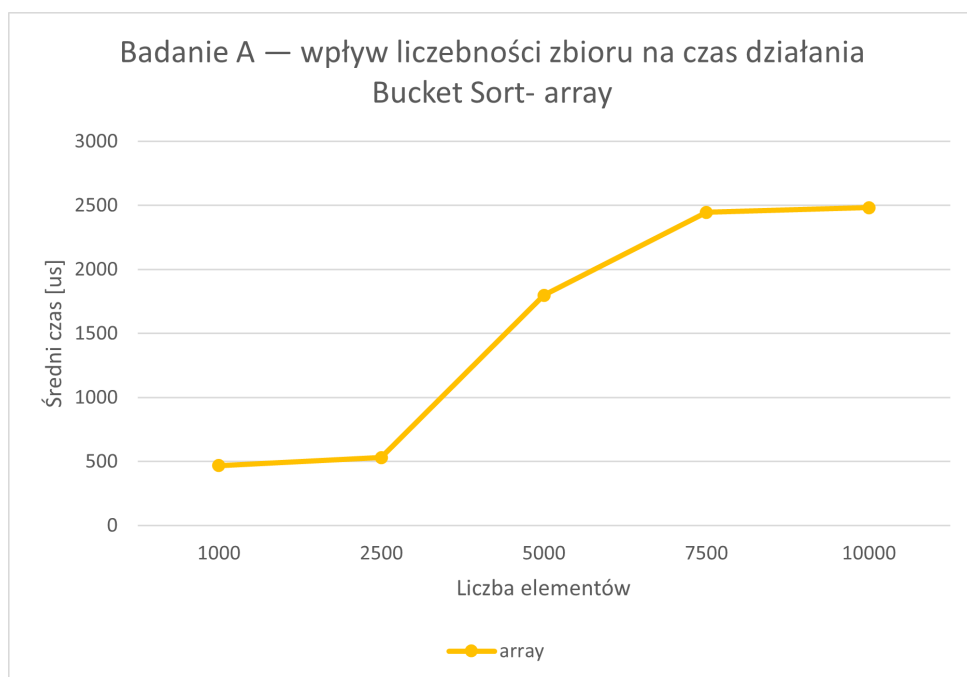
6.2.3 Bucket Sort

Tabela 5: Badanie A – Bucket Sort

Rozmiar	Array Avg [μs]	SingleList Avg [μs]	DoubleList Avg [μs]
1000	468.90	3816.13	3486.37
2500	530.47	57756.13	46595.33
5000	1797.37	118354.73	109396.87
7500	2447.10	247805.53	213164.23
10000	2482.57	364215.53	288694.53



Rysunek 6: Wykres ogólny Badanie A - Bucket Sort



Rysunek 7: Powiększenie dla mniejszych wartości - Array

Dla Bucket Sort najlepsze wyniki uzyskano dla tablicy. Różnica między tablicą a listami była bardzo duża, szczególnie dla większych rozmiarów danych. Wynika to z faktu, że dostęp do elementów w tablicy odbywa się bezpośrednio, natomiast w listach dostęp do elementu według indeksu wymaga przechodzenia przez kolejne węzły.

Lista dwukierunkowa w większości przypadków uzyskała nieco lepsze wyniki niż lista jednokierunkowa, jednak obie struktury były wyraźnie wolniejsze od tablicy.

6.2.4 Wniosek z badania A

Badanie A potwierdziło, że liczebność zbioru miała wyraźny wpływ na czas działania wszystkich analizowanych algorytmów. Wraz ze wzrostem liczby elementów czas sortowania rósł we wszystkich przypadkach, chociaż pojedyncze wartości średnie mogły być miejscami zaburzone przez warunki uruchomieniowe.

Najważniejszy wniosek dotyczy jednak rodzaju użytej struktury danych. We wszystkich badanych algorytmach najlepiej wypadła tablica, ponieważ zapewnia szybki dostęp do elementów po indeksie. Lista jednokierunkowa i dwukierunkowa osiągały wyraźnie gorsze wyniki, szczególnie w algorytmach wymagających częstego odczytu i zapisu elementów według pozycji.

Spośród porównywanych algorytmów najlepiej na strukturach listowych poradził sobie Bucket Sort. Było to szczególnie widoczne przy większych rozmiarach danych. Można więc uznać, że był on najmniej wrażliwy na użycie list, chociaż również w jego przypadku czasy działania dla list rosły szybciej niż dla tablicy.

Na podstawie badania A do kolejnych badań przyjęto rozmiar:

5000,

ponieważ stanowił wartość pośrednią: był wystarczająco duży, aby ujawnić różnice między strukturami i algorytmami, a jednocześnie pozwalał jeszcze wykonywać pomiary w akceptowalnym czasie w środowisku maszyny wirtualnej.

6.3 Badanie B – wpływ rozkładu danych

Celem badania B było określenie wpływu rozkładu danych wejściowych na czas działania algorytmu. Badanie wykonano dla Quick Sort z pivotem środkowym, dla trzech struktur liniowych:

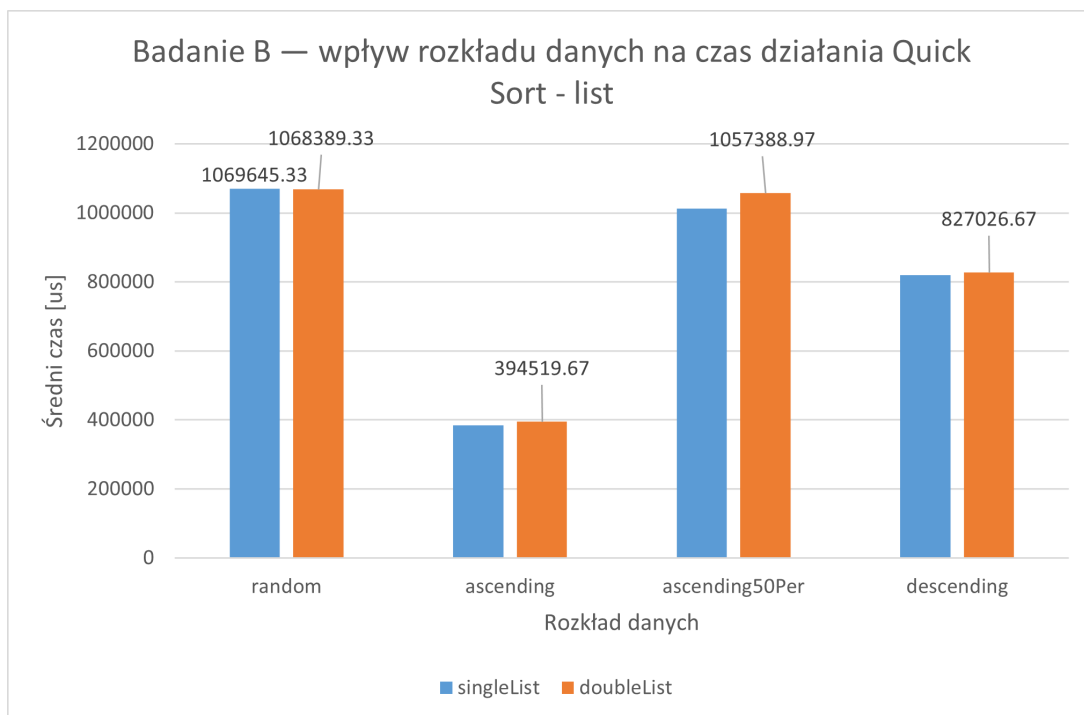
- Array,
- SingleList,
- DoubleList.

Użyto typu danych `int`, rozmiaru 5000 oraz 30 iteracji. Porównano rozkłady:

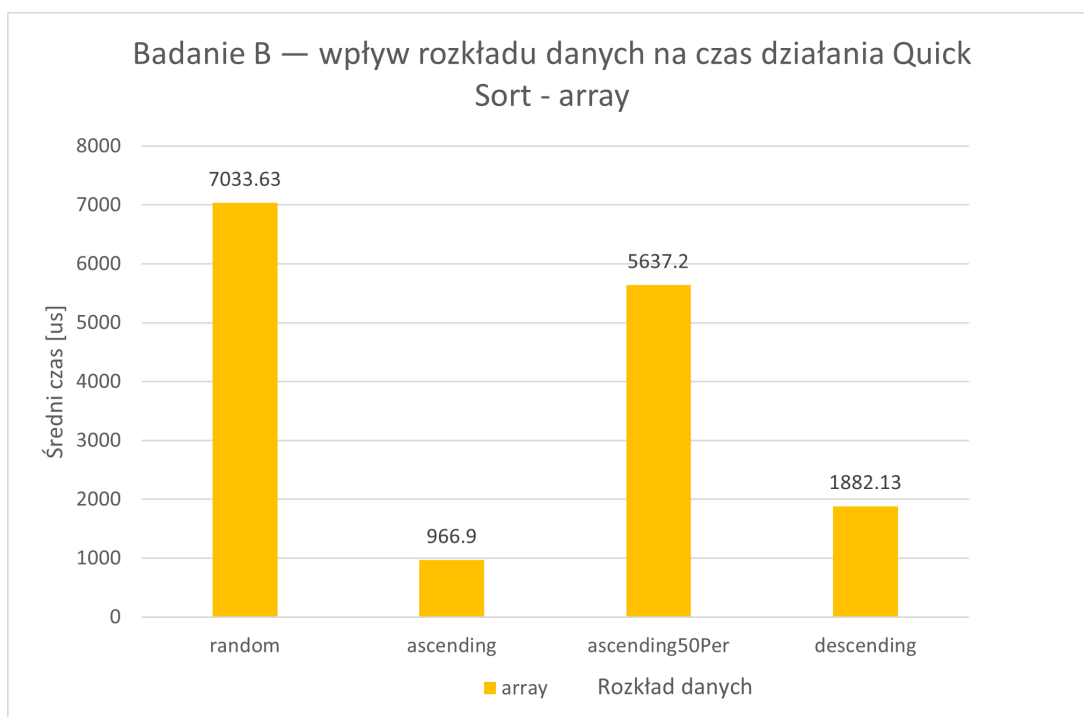
- random,
- ascending,
- descending,
- ascending50Per.

Tabela 6: Badanie B – wpływ rozkładu danych na czas działania Quick Sort

Rozkład	Array Avg [μs]	SingleList Avg [μs]	DoubleList Avg [μs]
Random	7033.63	1069645.33	1068389.33
Ascending	966.90	384645.63	394519.67
Descending	1882.13	819953.03	827026.67
Ascending50Per	5637.20	1012668.33	1057388.97



Rysunek 8: Wpływ rozkładu danych na czas działania Quick Sort - listy



Rysunek 9: Wpływ rozkładu danych na czas działania Quick Sort - Array

6.3.1 Wniosek z badania B

Badanie B pokazało, że rozkład danych wejściowych miał duży wpływ na czas działania Quick Sort. Najlepsze wyniki uzyskano dla przypadku **ascending**, trochę gorsze dla **descending**, natomiast najwięcej czasu algorytm potrzebował dla danych losowych oraz

częściowo uporządkowanych. Pokazuje to, że nawet przy takiej samej liczbie elementów, tym samym algorytmie i tej samej strukturze danych końcowy czas działania może się wyraźnie różnić tylko dlatego, że elementy są ułożone w inny sposób. Przy ocenie wydajności Quick Sort nie wystarczy brać pod uwagę jedynie rozmiaru danych, ale trzeba również uwzględnić ich początkowy układ, ponieważ może on w praktyce znacząco zmienić przebieg sortowania i wpłynąć na końcowy wynik pomiaru.

6.4 Badanie C – wpływ typu danych

Celem badania C było określenie wpływu typu danych na czas działania algorytmu. Badanie wykonano dla:

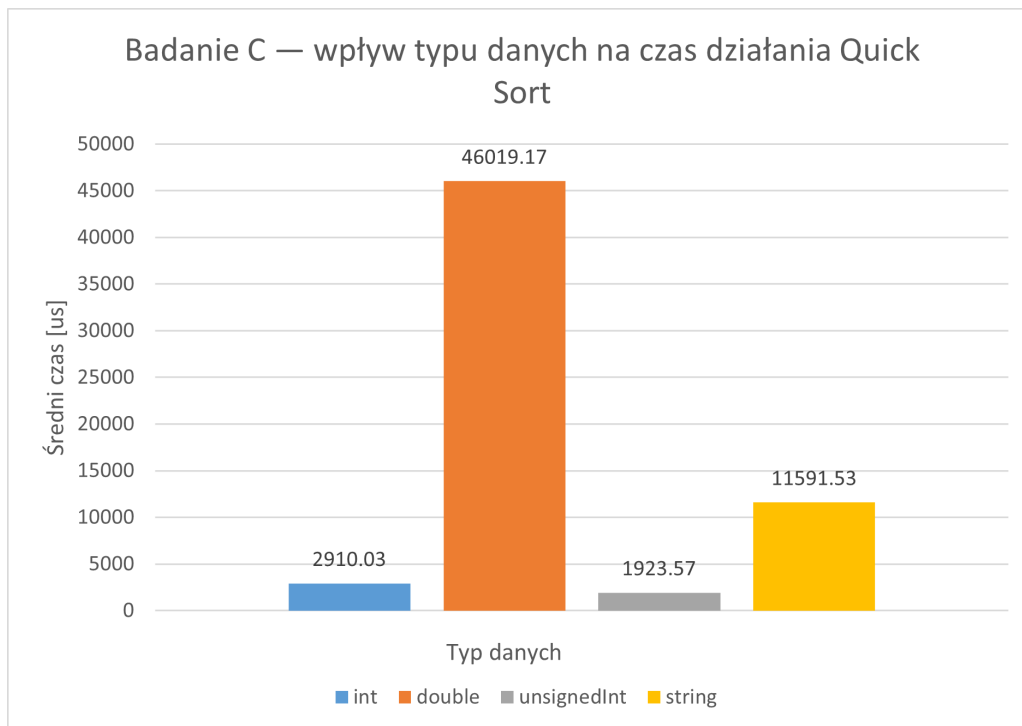
- algorytmu Quick Sort z pivotem środkowym,
- struktury `Array`,
- rozmiaru 5000,
- rozkładu `random`,
- 30 iteracji.

Porównano typy:

- `int`,
- `double`,
- `unsigned int`,
- `std::string`.

Tabela 7: Badanie C – wpływ typu danych na czas działania Quick Sort

Typ danych	Min [μ s]	Max [μ s]	Avg [μ s]
<code>int</code>	808	15921	2910.03
<code>double</code>	35439	123611	46019.17
<code>unsigned int</code>	760	6662	1923.57
<code>std::string</code>	5132	23523	11591.53



Rysunek 10: Wpływ typu danych na czas działania Quick Sort.

Szczególnie interesujący okazał się wynik dla typu `double`, który w badanej konfiguracji był wyraźnie wolniejszy od pozostałych typów. Najbardziej prawdopodobnym powodem nie jest sam fakt użycia typu zmiennoprzecinkowego, lecz sposób generowania danych testowych. W projekcie dla `double` wykorzystano bardzo szeroki zakres wartości, obejmujący niemal cały dopuszczalny przedział tego typu. Oznacza to, że algorytm pracował na danych o bardzo dużej skali liczbowej, co mogło negatywnie wpłynąć na stabilność i efektywność pomiarów. Dlatego wynik dla `double` należy interpretować ostrożnie: pokazuje on zachowanie programu w tej konkretnej implementacji generatora danych, a nie ogólną regułę, że typ `double` zawsze jest wolniejszy od innych typów.

6.4.1 Wniosek z badania C

Badanie C potwierdziło, że typ danych może silnie wpływać na czas działania algorytmu sortowania. Najszybsze okazały się typy całkowite, natomiast znacznie wolniejsze były `std::string` oraz szczególnie `double`. Oznacza to, że przy analizie wydajności algorytmu nie można ograniczać się wyłącznie do liczby elementów, lecz trzeba również uwzględnić właściwości porównywanych danych.

6.5 Badanie Ω – użycie nieliniowych struktur danych

Celem badania Ω było porównanie struktur liniowych z wybranymi strukturami nieliniowymi. W badaniu porównano:

- Array,
- SingleList,
- DoubleList,

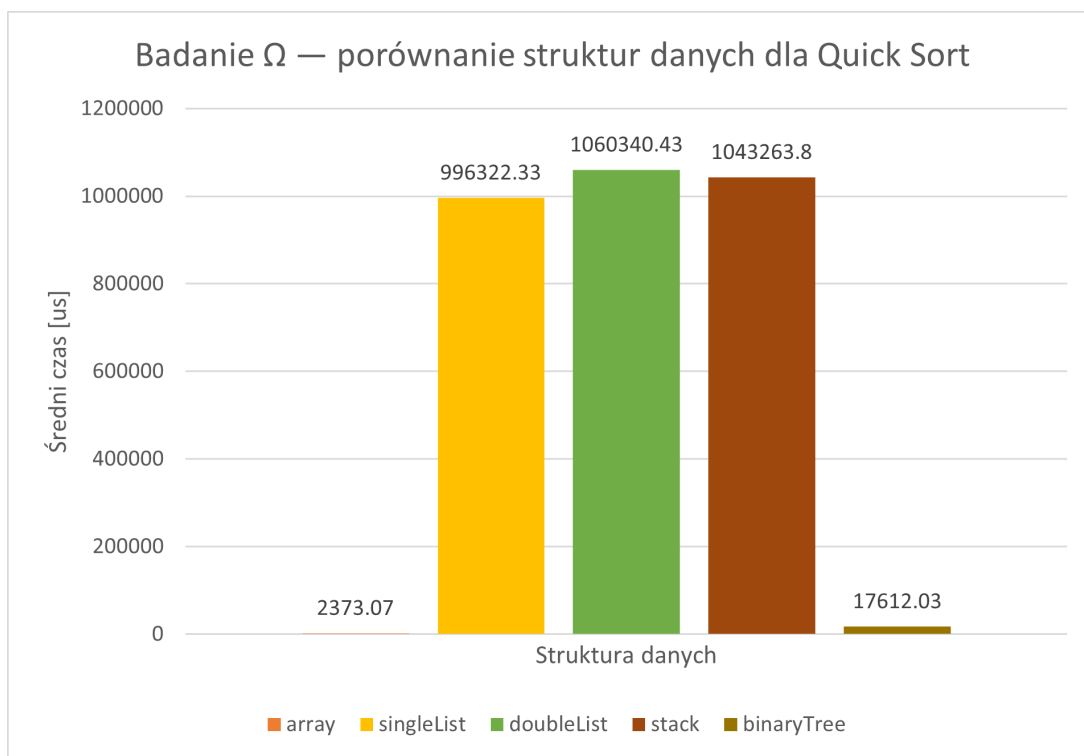
- Stack,
- BinaryTree.

Wykorzystano:

- Quick Sort z pivotem środkowym,
- typ danych int,
- rozkład random,
- rozmiar 5000,
- 30 iteracji.

Tabela 8: Badanie Ω – porównanie struktur liniowych i nieliniowych

Struktura	Min [μ s]	Max [μ s]	Avg [μ s]
Array	737	16638	2373.07
SingleList	622619	1323358	996322.33
DoubleList	692996	1390067	1060340.43
Stack	699163	1323189	1043263.80
BinaryTree	11544	29242	17612.03



Rysunek 11: Porównanie struktur danych dla Quick Sort w badaniu Ω .

6.5.1 Wniosek z badania Ω

Badanie Ω pokazało, że wybór struktury danych miał bardzo duży wpływ na wydajność analizowanego algorytmu. Zdecydowanie najlepiej ponownie wypadła tablica, która osiągnęła najkrótszy czas działania. Drzewo binarne zajęło pozycję pośrednią i okazało się wyraźnie lepsze od struktur liniowych opartych na przechodzeniu po kolejnych elementach. Najslabiej wypadły natomiast listy oraz stos, dla których czasy działania były zdecydowanie najwyższe. Wyniki te pokazują, że algorytm oparty na częstym dostępie do elementów według pozycji najlepiej współpracuje ze strukturą zapewniającą szybki i bezpośredni dostęp, a znacznie gorzej radzi sobie tam, gdzie konieczne jest przechodzenie przez kolejne węzły lub elementy.

7 Porównanie dwóch metodologii benchmarku

W trakcie realizacji projektu wykorzystano dwie wersje metodologii benchmarku. W pierwszej wersji przed rozpoczęciem serii pomiarowej tworzona była jedna struktura bazowa, a następnie przed każdą iteracją wykonywano jej kopię. Oznaczało to, że wszystkie powtórzenia w danej serii sortowały ten sam układ danych wejściowych. W wersji końcowej podejście to zostało zmienione: przed każdą iteracją tworzona była nowa struktura i ponownie wypełniana zgodnie z wybranym rozkładem danych. Dzięki temu każda iteracja dla rozkładu `random` korzystała z nowego zestawu danych losowych.

Porównanie obu metodologii okazało się szczególnie interesujące w badaniu α , ponieważ właśnie tam najłatwiej było zaobserwować wpływ zmiany sposobu przygotowywania danych wejściowych na końcowe wyniki.

7.1 Tabela porównawcza

Tabela 9: Porównanie dwóch metodologii benchmarku w badaniu α

Algorytm	Parametr	Średni czas przy kopiowaniu [μ s]	Średni czas w wersji końcowej [μ s]
Quick Sort	random	2200.37	1439.33
Quick Sort	middle	1836.40	776.00
Quick Sort	edge	2045.77	1369.60
Shell Sort	option1	2688.90	2629.60
Shell Sort	option2	2308.90	2239.47

Tabela pokazuje, że zmiana metodologii znacznie mocniej wpłynęła na wyniki Quick Sort niż Shell Sort. W przypadku Quick Sort różnice między wersją opartą na kopiowaniu jednej struktury bazowej a wersją końcową były dobrze widoczne. Może to oznaczać, że algorytm ten był bardziej wrażliwy na konkretny układ danych wejściowych. W przypadku Shell Sort średnie wartości pozostały do siebie znacznie bardziej zbliżone, co sugeruje większą stabilność tego algorytmu przy zmianach danych losowych pomiędzy kolejnymi iteracjami.

Na tej podstawie można stwierdzić, że wcześniejsza metodologia z kopiowaniem jednej struktury bazowej nie była całkowicie bezużyteczna. Pozwalała ona badać stabilność algorytmu dla jednego konkretnego przypadku wejściowego. Jednak z punktu widzenia końcowych badań projektowych bardziej właściwe okazało się generowanie nowych danych

przed każdą iteracją, ponieważ taka metoda lepiej odzwierciedla działanie algorytmu na rzeczywistym zbiorze wielu przypadków testowych.

7.2 Przykładowe polecenia uruchomienia programu

Poniżej przedstawiono przykładowe polecenia uruchomienia programu wykorzystane podczas testowania i wykonywania badań.

Tryb singleFile

```
./Project --singleFile --inputFile in.txt --outputFile out.txt -a 5 -p 3 -s 0 -t 0
```

Tryb benchmark – Quick Sort

```
./Project --benchmark -a 5 -p 0 -s 0 -t 0 -d 0 -l 5000 -n 30 -r ../analysis/data/results/alpha_quick.csv
```

Tryb benchmark – Shell Sort

```
./Project --benchmark -a 6 -e 0 -s 0 -t 0 -d 0 -l 5000 -n 30 -r ../analysis/data/results/alpha_shell.csv
```

8 Repozytorium GitHub

Projekt był rozwijany z wykorzystaniem repozytorium GitHub: <https://github.com/Cattein/Project.git>

9 Wnioski

Jeśli chodzi o analizę projektu, to okazało się, że nawet w przypadku zwykłego sortowania na końcowy wynik może wpływać bardzo wiele różnych czynników, takich jak dobrane parametry algorytmu, struktura danych, typ danych wejściowych czy ich początkowy układ.

Badanie α potwierdziło, że dobór parametrów algorytmu ma znaczenie. Najlepsze wyniki uzyskał Quick Sort z pivotem środkowym oraz Shell Sort z wariantem odstępów `option2`. Pokazuje to, że nawet niewielkie zmiany w sposobie działania algorytmu mogą wpływać na czas wykonania.

Jednym z najważniejszych wniosków okazała się analiza struktury danych. Już w momencie, gdy przy próbie generowania danych o rozmiarze 20000 dla list środowisko przestało działać stabilnie, było widać, że tablica wypada znacznie lepiej, albo że należałoby jeszcze raz bardzo dokładnie sprawdzić implementację. Po ponownej weryfikacji kodu nie znalazłam jednak istotnych błędów, a końcowe wyniki badań tylko to potwierdziły. We wszystkich głównych badaniach najlepiej wypadała tablica, ponieważ zapewnia szybki

dostęp do elementów po indeksie. Wśród analizowanych algorytmów Bucket Sort okazał się natomiast najmniej wrażliwy na użycie list.

Badanie B pokazało, że również rozkład danych wejściowych ma znaczenie. W przypadku Quick Sort najlepsze wyniki uzyskano dla danych rosnących, a słabsze dla danych losowych i częściowo uporządkowanych.

Z kolei badanie C potwierdziło, że wpływ ma także typ danych. Najlepiej wypadały typy całkowite, a wyraźnie gorzej `double` (dla bardzo szerokiego zakresu wartości) oraz `std::string`.

Badanie Ω jeszcze raz potwierdziło, że wybór struktury danych ma bardzo duży wpływ na końcowy wynik. Tablica ponownie była najlepsza, drzewo binarne zajęło pozycję pośrednią, a listy i stos wypadły najslabiej.

W trakcie realizacji projektu cały czas pojawiał się też inny problem, czyli niedostatek wiedzy. Na początku wymagań było naprawdę dużo i wydawały się trudne, jednak w miarę pracy nad projektem można było zauważyć, że wiele z tych rzeczy staje się zrozumiałych i logicznych. Pod koniec realizacji okazało się, że wiele rozwiązań jest prostszych, niż wydawało się na początku — nie w tym sensie, że wystarczy napisać „dwie linijki kodu”, ale raczej w tym, że można przygotować jedno bardziej uniwersalne rozwiązanie, na przykład szablon, a następnie wykorzystać je w wielu miejscach programu.

Drugim minusem projektu był, moim zdaniem, limit czasu. Trudno było pogodzić ten projekt z innymi przedmiotami, dlatego wiele pomysłów kończyło się niestety na stwierdzeniu, że brakuje czasu na ich pełną realizację. Chciałabym na przykład przygotować jeszcze ciekawszą analizę, bardziej rozbudować sposób porównywania wyników, dokładniej opracować metodę uśredniania lub przygotować dodatkowe podejście do interpretacji wartości minimalnych i maksymalnych. Interesującym pomysłem byłoby również napisanie skryptu albo prostego mini-aplikacji z oknem, w której można byłoby wybrać konkretne badanie, a program automatycznie generowałby wykres i podstawowe informacje potrzebne do analizy.

Jednak poza samymi wnioskami analitycznymi bardzo duże znaczenie miało również samo tworzenie projektu. Istotnym doświadczeniem okazało się przygotowanie własnej struktury programu, wykorzystanie szablonów, poprawne dołączanie bibliotek oraz organizacja całego kodu. Mimo że w trakcie pracy pojawiały się różne błędy, właśnie one pozwoliły lepiej zrozumieć, jak powinno się poprawnie budować i rozwijać większe projekty programistyczne. Dzięki temu projekt pokazał mi, że najważniejsze jest cały czas zadawać sobie pytania: „dlaczego?”, „po co?” oraz „jak można to zrobić lepiej?”.

Chciałabym również dodać, że niezależnie od tego, ile testów się wykona, nie zawsze da się zauważyć wszystkie błędy. Czasami można sprawdzić projekt wiele razy, a mimo to przeoczyć jedną pozornie drobną i głupią pomyłkę. Kiedy długo pracuje się nad projektem, a potem okazuje się, że całość „wykłada się” właśnie przez taki szczegół, trudno się nie załamać.

Z drugiej strony jest to naprawdę cenne doświadczenie. Lepiej popełnić taki błąd w projekcie na studiach niż w prawdziwym systemie, w którym taka pomyłka mogłaby spowodować poważne straty. Dzięki temu można lepiej zrozumieć, jak ważne jest dokładne sprawdzanie kodu, parametrów, wyników oraz zgodności programu z wymaganiami.

Korzystając z okazji, chciałabym też zaznaczyć, że aby unikać takich błędów, warto sprawdzać projekt nie tylko własnym spojrzeniem, ale również poprosić drugą osobę o krótką weryfikację. Nie chodzi o to, żeby ktoś wykonywał projekt za nas, ale o to, żeby sprawdził, czy wszystko się zgadza i czy nie ma oczywistych niespójności. Jedna głowa jest dobra, ale dwie często zauważą więcej.

Literatura

- [1] Algorytm.edu.pl. Czytanie pliku. <https://www.algorytm.edu.pl/pliki-tekstowe/czytanie-pliku.html>.
- [2] CodeSignal. Parsing csv files and converting strings to integers in c++. <https://codesignal.com/learn/courses/parsing-table-data-in-cpp/lessons/parsing-csv-files-and-converting-strings-to-integers-in-cpp>.
- [3] cpp0x.pl. Wskaźniki. <https://cpp0x.pl/kursy/Kurs-C++/Dodatkowe-materialy/Wskazniki/304>.
- [4] cppreference.com. C++ numeric library – random. <https://en.cppreference.com/cpp/numeric/random>.
- [5] Eduinf. Algorytmy sortowania. https://eduinf.waw.pl/inf/alg/003_sort/0020.php.
- [6] GeeksforGeeks. Bucket sort. <https://www.geeksforgeeks.org/dsa/bucket-sort-2/>.
- [7] GeeksforGeeks. Creating array of pointers in c++. <https://www.geeksforgeeks.org/cpp/creating-array-of-pointers-in-cpp/>.
- [8] GeeksforGeeks. Csv file management using c++. <https://www.geeksforgeeks.org/cpp/csv-file-management-using-c/>.
- [9] InfoTraining. Pointers and arrays. <https://infotraining.bitbucket.io/cpp-bs/pointers-arrays.html>.
- [10] D. Mroziński. Repozytorium z biblioteką parametrów dla studentów. https://gitlab.com/mrozo94/for-students/-/tree/master/sem_2025-2026?ref_type=heads.